

# DPRFuzz: Enhancing Vulnerability Mining With Two-Stage Reinforcement Learning

Liqun Yang<sup>1</sup>, Ruihao Li<sup>1</sup>, Chaoren Wei<sup>1</sup>, Jian Yang<sup>1</sup>, Yuze Yang<sup>1</sup>, Liang Sun<sup>1</sup>,  
Dong Zhao<sup>1</sup>, Senior Member, IEEE, and Zhoujun Li<sup>1</sup>

**Abstract**—American fuzzy lop (AFL), as a representative tool for fuzzing, is capable of uncovering security vulnerabilities in industrial systems. It suffers from consuming a large amount of computational resources during the mutation. To improve the performance of AFL, researchers adopt algorithms, such as particle swarm optimization and long short-term memory, to optimize mutation operator selection. However, challenges persist in these approaches integrated with AFL, including optimization model complexity, insufficient accuracy, and poor generalization scalability. To address these issues, the article proposes a new fuzzer called DPRFuzz to optimize AFL's mutation phases. First, in the deterministic mutation strategy mutation phase, deep Q network and trust region policy optimization are leveraged to precisely generate effective mutated samples through perceiving mutation process in a relatively short time. Then, to boost the efficiency of the Havoc random mutation phase, we improve the Thompson sampling algorithm based on a multiagent strategy to generate an overall optimal mutation strategy chain. Finally, the approach is tested on eight programs, such as *readelf*, *tcpdump*, and *nm*, and the advantages of DPRFuzz are analyzed. Most importantly, the experiment reveals results that DPRFuzz achieves better fuzzing performance compared to the traditional and other AFL-based fuzzers, such as AFL, AFL++, AFLSmart, etc. On average, DPRFuzz achieves an improvement in code coverage of over 10%, along with a significant increase in the number of crashes.

**Index Terms**—American fuzzy lop (AFL), deep reinforcement learning (RL), fuzzing, optimization, vulnerability discovery.

## I. INTRODUCTION

WITH the development of information technology, software becomes increasingly complex and vulnerable [1], [2], [3]. While traditional security testing methods are effective in identifying certain vulnerabilities, they fall short in detecting subtle flaws due to their limitations in exploring the deep-seated execution paths. As an effective vulnerability mining technique, fuzzing addresses the shortcomings of traditional security testing methods, enhancing the effectiveness of vulnerability mining in the industrial field [4], [5].

American fuzzy lop (AFL) utilizes a coverage-guided paradigm and combines dynamic instrumentation of execution paths to enhance code coverage and vulnerability discovery performance [6], [7], [8]. It is widely employed in security testing tasks, uncovering numerous security vulnerabilities in industrial software, such as OpenPLC, Beremiz, and AutoCAD. Besides, AFL is also renowned for its lightweight and efficient design, which is a popular fuzzing choice for researchers. In the AFL mutation process, it monitors different program execution paths to guide the generation of mutated samples, which trigger new code execution paths [9]. AFL-fuzz file is the critical component of AFL, in which the mutation function is composed of multiple phases, such as calibration, mutation strategies, Havoc, etc. The relationship between these phases is illustrated in Fig. 1.

Although AFL achieves outstanding performance in fuzzing, several urgent issues still need to be addressed. First, the randomness of AFL's mutated sample generation will lead to a decrease in fuzzing efficiency, making it difficult to rapidly uncover vulnerabilities. Second, AFL has certain limitations in discovering complex vulnerabilities and logical errors. Researchers improve AFL through symbolic execution and taint analysis, resulting in several optimized versions, including AFLFast and CollAFL [10], [11]. But these optimized versions still face issues of low resource utilization and poor vulnerability detection [12], [13]. To address these issues, the AFL core mutation phase is divided into the deterministic strategies mutation phase and the Havoc random mutation phase. In the deterministic mutation strategy mutation phase, mutation positions are selected for mutation under fixed mutation strategies. In the Havoc random mutation phase, multiple mutations are performed after

Received 7 May 2025; revised 13 July 2025; accepted 13 July 2025. Date of publication 19 August 2025; date of current version 17 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62302025, Grant U2333205, Grant 62273021, and Grant U24A20243, in part by the a fund project: State Grid Co., Ltd. Technology RD Project under the Project Name: Research on Key Technologies of Data Scenario-based Security Governance and Emergency Blocking in Power Monitoring System, under Grant 5108-202303439A-3-2-ZN, and in part by the Beijing Natural Science Foundation under Grant 4232047. Paper no. TII-25-2991. (Corresponding author: Dong Zhao.)

Liqun Yang, Chaoren Wei, Yuze Yang, Liang Sun, and Dong Zhao are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: lqyang@buaa.edu.cn; weichaoren@buaa.edu.cn; yangyuze@buaa.edu.cn; eel-sun@buaa.edu.cn; dzhao@buaa.edu.cn).

Ruihao Li is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 102206, China (e-mail: ruihao.li@bupt.edu.cn).

Jian Yang and Zhoujun Li are with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: jiaa@buaa.edu.cn; lijz@buaa.edu.cn).

Digital Object Identifier 10.1109/TII.2025.3593913

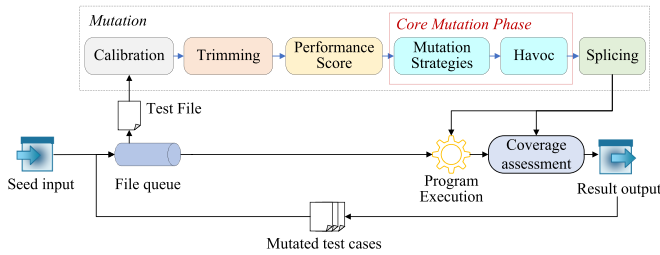


Fig. 1. Flowchart of AFL execution.

forming mutation strategy chains based on a series of randomly selected mutation strategies. Our work employs reinforcement learning (RL) methods to select mutation operators, which consist of mutation positions and mutation strategies in the deterministic mutation phase and the mutation strategy chains in the Havoc random mutation phase. By this means, we can learn and make decisions based on high-dimensional perceptual inputs in discrete spaces, maximize the number of effective mutated samples and thereby, uncover deep-level vulnerabilities.

In the above two mutation phases, AFL employs blind fuzzing strategies that rely on random mutations, resulting in generating massive invalid test cases [14]. In this article, for the former phase, a value- and policy-based method is proposed to address the issue of low mutation efficiency caused by the exhaustive strategy under a fixed mutation policy. By learning the mutation process of AFL, this method can predict effective mutation operators that may trigger new paths and reduce the number of mutation rounds [15]. For the latter mutation phase, a lightweight decision-making method of selecting mutation strategies based on exploration and exploitation is proposed, which addresses the issue of computational resource waste caused by the random selection of strategies. By deploying the lightweight model in the Havoc phase and interacting with the mutation process, the predicted mutation strategy chains are likely to produce effective mutation strategies [16].

For evaluation, DPRFUZZ is trained based on eight programs and assesses the performance by four indicators. This article investigates the optimization effect of different RL models on AFL and further verifies the effectiveness in predicting mutation operators. The experimental results show that DPRFUZZ can significantly improve AFL fuzzing performance. In particular, DPRFUZZ outperforms the previous fuzzers, such as AFL, AFL++, and AFLSmart. Given the computational overhead challenge of integrating the RL models into AFL, the experiments are also conducted to comprehensively evaluate the resource cost of the models. Moreover, our research successfully identifies five zero-day vulnerabilities, validating the efficacy and innovation of our methodology.

DPRFUZZ, as a coverage-guided fuzzing method that improves path exploration and crash discovery at a lower computational cost, outperforms state-of-the-art fuzzers like AFL, while also demonstrating the value of joint optimization in mutation operators. In summary, this article makes the following contributions.

- 1) To the best of our knowledge, this is the first study to refine the mutation process of AFL into the deterministic

mutation phase and the Havoc random mutation phase, comprehensively improving the selection of effective mutation operators during the AFL mutation process. DPRFUZZ also proves effectiveness in real-world applications, and discovers five zero-day vulnerabilities.

- 2) We propose value- and policy-oriented models to generate effective mutation operators in the deterministic mutation strategy mutation phase. This addresses the blindness in selecting mutation strategies and the inefficiency in generating mutated samples for highly structured data.
- 3) We utilize a lightweight explore and exploitation strategy to optimize the mutation strategy chain construction in the Havoc random mutation phase, which generates more effective mutated samples under a fixed number of mutations for each test case and enhances fuzzing effectiveness.

The rest of this article is organized as follows. Section II briefly reviews the relevant research and provides the motivation. Section III introduces the system description and optimization problem definition. Section IV details the design of DPRFUZZ. Section V implements DPRFUZZ and demonstrates the experiments. Finally, Section VI concludes this article.

## II. RELATED WORKS

Fuzzing optimization techniques can predict mutation operators that yield the greatest benefits [17], [18]. Fuzzing optimization techniques include the following: 1) Formal verification and software analysis based techniques, utilizing symbolic execution and constraint-solving methods to optimize the generation of mutated samples, so as to explore more program execution branches [19]; 2) shallow learning aided techniques, integrating shallow learning algorithms into fuzzing frameworks to achieve efficient mutated sample generation based on prior knowledge [20]; 3) deep learning based techniques, employing deep learning models to learn the spatial distribution of test cases and generate more targeted and diverse test cases, cementing the effectiveness of fuzzing [21], [22], [23].

RL methods can be classified into policy-based methods, deep RL methods and exploration-exploitation-based methods, etc. Deep RL methods introduce deep neural networks to handle high-dimensional complex inputs and action spaces. The representative methods include deep Q network (DQN), trust region policy optimization (TRPO), etc [24]. The exploration-exploitation-based methods balance the exploration of new policies and the utilization of known policies to achieve high learning efficiency and quality decision-making [25], [26]. Typical methods include greedy algorithms, upper confidence bound algorithms, and Thompson sampling (TS) algorithms [27], [28].

In addition, researchers try to improve fuzzers to adapt to different fuzzing tasks [29], [30]. Specifically, to meet the requirements of parallel fuzzing, Zhou et al. [31] designed a distributed fuzzer UltraFuzz. By implementing centralized dynamic scheduling, UltraFuzz allocates test assignments that avoid resource waste caused by the random generation of test cases [31]. Li et al. [32] introduced contrastive curriculum learning into DL-based vulnerability detection to find a suitable boundary to distinguish vulnerabilities from normal code. They

select four state-of-the-art detectors as models and improve their F1 scores by 8.1% on a real-world dataset [32]. Zhang et al. [33] proposed Gen-Fuzz, a GPU-accelerated fuzzer utilizing a genetic algorithm for multi-input fuzzing. It achieves an 80x runtime speedup compared to state-of-the-art fuzzers. Gen-Fuzz significantly accelerates fuzzing, but the GPU service efficiency is still a problem to break through [33].

Furthermore, to expand the ability of detecting deep-seated vulnerabilities, Zhu et al. [34] proposed CSI-Fuzz, an instrumentation-based fuzzer that solves edge collision issues in full-speed fuzzing. By directly instrumenting at edges and counting path frequencies, it outperforms AFL, AFLFast, and UnTracer in discovering more edges and exposing more bugs [34]. To address the challenge of optimal resource management and caching in ultra-reliable and low-latency communication-enabled industrial networks, Hazarika et al. [35] proposed a framework integrating Mobile Edge Computing servers with Roadside Units, leveraging AFL for global model accuracy and multiagent actor-critic deep RL-based algorithm for resource allocation optimization. It also utilizes a cooperative caching scheme for content prediction and improves efficiency [35]. Zhu et al. [36] proposed ATTuzz to address the limitations of existing fuzzers by utilizing an attention integrated neural network to estimate block rewards for seed selection, while it relies on frequent model updates and requires substantial computational resources [36]. Eom et al. [37] combined large language models with RL for fuzzing, improving input validity, but lacking semantic-aware guidance [37]. Tappler et al. [38] proposed a framework that combines search-based testing with RL, but suffers from unaddressed generalization challenges [38].

The aforementioned studies still encounter issues related to fuzzing efficiency and resource limitations, as well as challenges in technology integration and application. Specifically, the selection of effective mutation operators in fuzzing remains a pivotal factor that determines the efficacy of fuzzing. In this article, our method mitigates model consumption and achieves great generalization capabilities across various programs. This article also modularizes the optimization models and renders them easily deployable in diverse application scenarios.

### III. SYSTEM DESCRIPTION AND PROBLEM DEFINITION

In this article, the fuzzing process is defined as the environment  $E$ , the mutation strategy the RL model outputs as the strategy  $\pi$ , and the current test case as the state  $S$ . In mutation, the effectiveness of a mutated sample is defined as the reward  $r$ . If the mutated sample triggers a new path, then  $r = 1$ , otherwise,  $r = 0$ . The selected mutation operator is defined as the action  $a$  and the number of effective mutated samples as the total reward  $r_{sum}$ . Specifically, the test case is defined as a sequence of bytes  $S = (x_1, \dots, x_n)$ . Then, the optimization selection of mutation operators in AFL can be defined as follows:

$$r_{sum} = \sum r = \sum_{r=0}^n \max_a E(M(S)) \quad (1)$$

where  $M$  is the mutation process of AFL, and  $n$  is the number of mutation rounds.

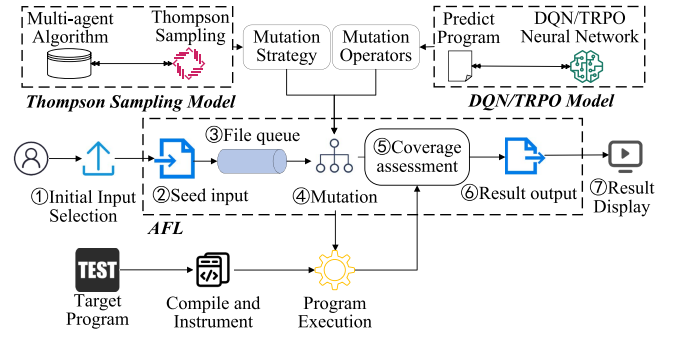


Fig. 2. Flow chart of AFL optimization in different phases.

In the deterministic mutation strategy mutation phase, the mutation operators are defined as the mutation strategy-mutation position pair  $(o, p)$  and describe the model's output strategy as  $\pi$ . Then, the relationship between the model's output strategy and the mutation operators can be defined as  $\pi = [(o_1, p_1), (o_2, p_2), \dots, (o_n, p_n)]$ . Here,  $o$  is the mutation strategy with a total of 16 values,  $p$  is the mutation position, and  $n$  is the total number of model predictions.

In the Havoc random mutation phase, AFL randomly generates mutation strategy chains to significantly modify the test case and trigger new paths. In this phase, the mutation strategy is defined as the action  $a^*$ , the model's output mutation strategy chain as the strategy  $\pi^*$ , and the relationship between them as  $\pi^* = [a_1^*, a_2^*, \dots, a_n^*]$ . The two-phase optimization for AFL can be summarized as Fig. 2. Aside from the definitions above, prior knowledge plays a crucial role in better understanding of the RL methods, and it will be introduced as follows.

For the DQN algorithm, it calculates the Q-value through the action-value function  $Q_\pi(S, a)$  to determine the expected reward  $r$ , when taking action  $a$  under policy  $\pi$  and state  $S$ . It makes globally optimal decisions based on the magnitude of the Q-value, which can be seen as the action-value function under the optimal policy. DQN achieves the fitting of the action-value function in the current environment through value function iteration, which can be defined as follows:

$$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha(r_t + \gamma \cdot \max_{a_{t+1}} Q(S_{t+1}, a_{t+1}) - Q(S_t, a_t)) \quad (2)$$

where  $\alpha$  and  $\gamma$  are the learning rate and the discount factor, respectively.  $r$  is the reward function and  $t$  is the number of iterations of the DQN model.  $Q(S_t, a_t)$  is the Q-value of taking action  $a_t$  in state  $S_t$  at time  $t$ .

For the TRPO algorithm, it defines the discount function to evaluate the quality of the strategy for selecting mutation operators. The discount function is defined as follows:

$$\eta(\pi) = E_{s_0, a_0, \dots} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (3)$$

where  $E$  is expectation,  $\gamma$  is the parameter set by the algorithm and  $r$  is the reward the mutation gains.

Subsequently, to ensure that the updated policy is superior to the old one, the discount function of the updated policy  $\pi'$  is computed as follows:

$$\eta(\pi') = \eta(\pi) + \sum_S \rho'_\pi(a|S) \sum_a \pi'(a|S) A_\pi(S, a) \quad (4)$$

where  $\rho$  and  $A$  are the discounted visit frequency and the advantage function, respectively. A nonnegative term  $\sum_S \rho'_\pi(a|S) \sum_a \pi'(a|S) A_\pi(S, a)$  is generated based on (4). Since the discount function of the new policy is the sum of that of the old policy and the nonnegative term, the new policy is superior to the old one.

#### IV. DPRFUZZ

This article leverages the advantages of different RL methods to address the complex optimization challenges of the mutation phases. Concretely, based on the action-value function structure, DQN enables efficient model training and deployment, reducing the randomness in policy selection within high-dimensional state spaces. In view of the TRPO model constraining the policy function via KL divergence, it not only enhances the model's robustness, but also ensures that fuzzing outputs stable mutation strategies. The TS algorithm, with its low parameter maintenance cost and strong adaptability, effectively mitigates the computational overhead caused by frequent data interactions in the fuzzing process.

##### A. DQN-Oriented Mutation Efficiency Optimization

Our work transforms the deterministic mutation strategy mutation phase into a Markov chain and proposes a mutation operator selection model based on the DQN algorithm, which decides the mutation operators to improve mutation efficiency. Compared with traditional Q-learning, which is difficult to handle high-dimensional test cases, DQN utilizes deep neural networks to resolve the issue.

Based on the mentioned prior knowledge, the gradient descent method is used to adjust neural network parameters  $\theta$  and minimize the loss function of the DQN model, aiming to approximate the optimal action-value function. To minimize the value of  $r_t + \gamma \cdot \max_a Q(S_{t+1}, a) - Q(S_t, a_t)$ , the loss function can be defined as follows:

$$L(\theta) = E((r + \gamma \cdot \max_{a_{t+1}} Q(S_{t+1}, a_{t+1}; \theta_i^-) - Q(S_t, a_t; \theta_i^-))^2) \quad (5)$$

where  $\theta_i$  is the target network parameter obtained during the  $i$ th iteration, and  $\theta_i^-$  is the value network parameter. Therefore, the loss function is approximately expressed as follows:

$$L(\theta) \approx \frac{1}{2} [Q(S_t, a_t; \theta_i) - [r + \gamma \cdot \max_{a_{t+1}} Q(S_{t+1}, a_{t+1}; \theta_i^-)]]^2 \quad (6)$$

where  $Q$  is the Q-value function,  $S$  is the current test case,  $a$  is the mutation operators, and  $r$  is the effectiveness of current mutations. After a certain number of mutations, DQN transfers  $\theta$  from the value network to update the target network, which ensures that the model maintains constant Q-values for

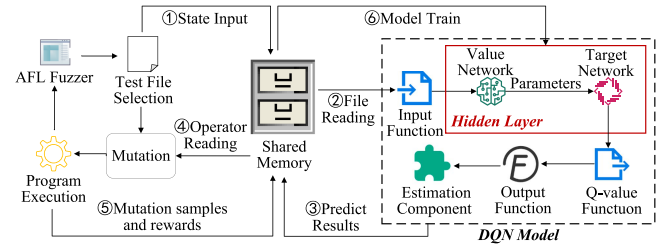


Fig. 3. Interaction process between AFL and DQN.

a specific duration. Meanwhile, DQN employs the experience replay mechanism to mitigate the impact of strong correlations among test cases on the model's policy selection effectiveness. The experience replay mechanism utilizes storage and sampling methods to break the tendency of test cases toward selecting mutation operators in a time series by designing a fixed-size replay memory unit to store experiences. The experiences are defined as quadruplets containing test cases, mutation operators, rewards, and mutated samples. The process of DQN predicting and selecting mutation operators for the test case, and the interaction process between AFL and DQN is illustrated in Fig. 3.

Considering that AFL generates multiple effective mutation operators that trigger new paths while traditional DQN can only output one prediction. This article innovatively designs a policy output function as an estimation component to ensure that DQN predicts and outputs multiple mutation operators. The principle of the estimation component is defined as follows:

$$U = p_1, p_2, \dots, p_k$$

$$Q(S_t, a_{p_1}) > Q(S_t, a_{p_2}) > \dots > Q(S_t, a_{p_n}) \quad (7)$$

where  $\pi$  is the policy, which represents the set of all mutation operators, and  $n$  is the number of mutation strategies.  $U$  is the set of mutation operators selected by the model and  $k$  is the number of selected mutation strategies.  $p$  is the index of the operator corresponding to the sorted Q-value in the Q-value table, and  $a_{p_k}$  is the action  $a$  that ranks  $k$ th in descending order of Q-values. The process of optimizing AFL based on DQN is presented in Algorithm 1.

##### B. TRPO-Oriented Mutation Efficiency Optimization

In the optimization of AFL with DQN, minor changes in the optimal Q-values will lead to deviations in selecting mutation operators. This results in the loss of AFL's ability to choose effective mutation operators. Furthermore, although DQN utilizes the experience replay mechanism to train the model, the utilization efficiency of experiences is low, which leads to insufficient model training. In this article, an optimization method is proposed for selecting mutation operators under a deterministic policy based on TRPO to resolve these issues. Compared to DQN, which utilizes an action-value function to predict actions, TRPO directly predicts mutation operators, enabling the neural network to output the policy function  $\pi(S)$  and determine the mutation operator under state  $S$  based on the maximum predicted policy function  $\max_a \pi(S)$ . The parameters in the TRPO

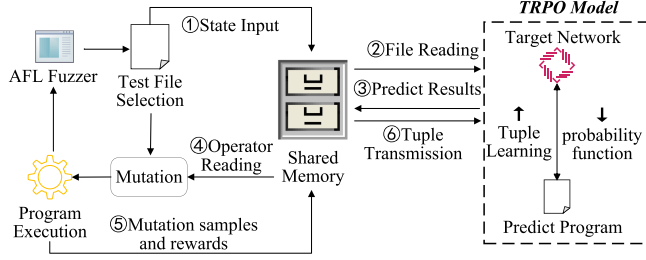


Fig. 4. Interaction process between AFL and TRPO.

**Algorithm 1: AFL Optimization Based on DQN model.****Input:** The program  $P$ , the seed of test cases  $S_0$ **Output:** The total reward  $r_{sum}$ , the trained model  $\mathcal{M}_{dqn}$ 

```

1: Embedding models in AFL
2:  $Q = S_0$ 
3: for  $S$  in  $Q$  do
4:    $\{(o, p)\}_{i=1}^N \leftarrow \mathcal{M}_{dqn}(S)$ 
5:   for  $(o, p)$  in  $\{(o, p)\}_{i=1}^N$  do
6:      $M \leftarrow \mathcal{F}(o, p, S)$ 
7:      $r \leftarrow P(M)$ 
8:     if  $r$  is success then
9:        $Q \leftarrow Q \cup M$ 
10:    end if
11:     $\text{update}(\mathcal{M}_{dqn}(S, (o, p), r, S'))$ 
12:  end for
13: end for

```

model are updated as follows:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} L(\theta_t) \quad (8)$$

where  $\alpha$  is the update size,  $L$  is the cost function, and  $\theta$  is the parameter of the model.

Based on the mentioned prior knowledge, the goal of the TRPO is to maximize the value of the cost function which can be approximated as follows:

$$L_{\pi}(\pi') = \eta(\pi) + \sum_S \rho_{\pi}(S) \sum_a \pi'(a|S) A_{\pi}(S, a) \quad (9)$$

where  $L$  is the cost function,  $\pi$  is the current strategy, and  $\pi'$  is the next better strategy.

Based on the state distribution under the old policy, the change trends of the cost function and the discount function are consistent when the parameter change is small. Therefore, the Kullback–Leibler (KL) divergence is introduced to constrain the similarity between the new and old policies [39], which prevents the new policy gradient from being too large and ensures the validity of the above approximation formula. The interaction between AFL and TRPO is illustrated in Fig. 4.

In the TRPO prediction, mutation operators with higher policy probability function values are selected for mutation, which is similar to (7). When updating the TRPO network, it does not consider the mutated samples, but only uses the current state  $S$ , the current mutation operator probability  $\pi$ , and the reward weighted sum  $r_{sum}$  calculated from the mutation operator probabilities

**Algorithm 2: AFL Optimization Based on the TRPO model.****Input:** The program  $P$ , the seed of test cases  $S_0$ **Output:** The total reward  $r_{sum}$ , the trained model  $\mathcal{M}_{trpo}$ 

```

1: Embedding models in AFL
2:  $Q = S_0$ 
3: for  $S$  in  $Q$  do
4:    $\{(o, p)\}_{i=1}^N \leftarrow \mathcal{M}_{trpo}(S)$ 
5:   for  $(o, p)$  in  $\{(o, p)\}_{i=1}^N$  do
6:      $M \leftarrow \mathcal{F}(o, p, S)$ 
7:      $r \leftarrow P(M)$ 
8:     if  $r$  is success then
9:        $Q \leftarrow Q \cup M$ 
10:    end if
11:     $\text{update}(\mathcal{M}_{trpo}(S, \pi, r))$ 
12:  end for
13: end for

```

and rewards. The parameter updating process is computed as follows:

$$\theta = R(S, \pi, r_{sum}) \quad (10)$$

where  $\theta$  is the parameter of the TRPO and  $R$  is the updating function. The process of optimizing AFL based on TRPO is shown in Algorithm 2.

**C. Improved TS-Oriented Mutation Effectiveness Optimization**

DQN and TRPO utilize the function values to guide the selection of mutation operators under specific states. However, in scenarios with continuous output predictions and a large number of states, issues, such as low learning efficiency and high storage requirements arise.

In addition, due to the relatively small number of mutations in the Havoc random mutation phase, the TS algorithm fails to learn the environment and to provide effective mutation strategy predictions in a short period. This article proposes an improved multiagent-based TS algorithm that acquires mutation strategy parameters from the mutation strategy posterior distribution. It can effectively overcome the issues of excessive storage requirements and sensitivity to the environment. In the Havoc random mutation phase, the selection of mutation strategies can be viewed as a multiarmed bandit problem, and each strategy can be considered as a different bandit

$$i_{best} = \arg \max \theta_i \quad (11)$$

where  $i$  and  $\theta$  are the index of the selected mutation strategy and the empirical probability of mutation strategy, respectively.

Define the effectiveness rate of mutation strategies predicted by the improved TS model as  $\sigma$  and the probability of the model selecting mutation strategy  $p_i$  is calculated as follows:

$$p_i = \sigma_i / \sum_{k=1}^K \sigma_k \quad (12)$$

where  $p$  represents the probability of selecting a mutation strategy and  $K$  indicates the number of mutation strategies.

**Algorithm 3:** AFL Optimization Based on Improved TS.

**Input:** The program  $P$ , the seed of test cases  $S_0$ , the constant  $m$ , the random function  $\mathcal{R}$ , the agent  $A$

**Output:** The total reward  $r_{sum}$ , the trained model

```

 $\mathcal{M}_{thompson}$ 
1: Embedding models in AFL
2:  $Q = S_0$ 
3: for  $S$  in  $Q$  do
4:   for  $r < m$  do
5:      $N \leftarrow \mathcal{R}(S)$ 
6:      $\{o\}_{i=1}^N \leftarrow \mathcal{M}_{ts}(N, A_1, A_2, \dots, A_n)$ 
7:     for  $o$  in  $\{o\}_{i=1}^N$  do
8:        $M \leftarrow \mathcal{F}_{(op, pos, S)}$ 
9:        $r \leftarrow P(M)$ 
10:      if  $r$  is success then
11:         $Q \leftarrow Q \cup M$ 
12:      end if
13:      update( $\mathcal{M}_{ts}(r, done)$ )
14:    end for
15:  end for
16: end for

```

In the optimization of mutation strategy selection, our work defines  $D_t$  as the mutation strategy-reward set of the first  $t$  rounds of mutation. The TS algorithm calculates the posterior distribution based on  $D_t$  and prior knowledge to predict the optimal mutation strategy chain. It is assumed that the effective probability of each mutation strategy follows the beta distribution, which consists of prior probability parameters  $\alpha$  and  $\beta$ . In AFL, the sets of  $\alpha$  and  $\beta$  for the beta distribution of mutation strategies are denoted as  $S[k]$  and  $F[k]$ , which are used to record the number of effective and ineffective mutated samples, where  $k$  is the index of the mutation strategy.

In the mutation process of test cases, the rewards of mutation operators follow a beta distribution, with the parameter distribution given by the following formula:

$$\eta \sim \text{beta}(\alpha, \beta) \quad (13)$$

where  $\eta$  is the reward probability parameter for each mutation strategy in the  $t + 1$  round of mutation.  $\alpha$  and  $\beta$  are positive real numbers. In the TS algorithm, the following holds:

$$\pi(\eta) \propto \eta^{\alpha-1}(1 - \eta)^{\beta-1} \quad (14)$$

where  $\pi(\eta)$  is the prior probability distribution of the mutation operator. Since the mutation operators are selected randomly, it is assumed that each mutation operator is independent. Thus, the posterior distribution of each mutation operator will be updated independently, resulting in the likelihood function of  $\eta$  at round  $t$  as follows:

$$P(S[k], F[k]|\eta) = \eta^{S[k]}(1 - \eta)^{F[k]} \quad (15)$$

where  $S$  is the number of effective mutations,  $F$  is the number of ineffective mutations,  $k$  is the sequence number of mutation strategy, and  $P$  is the likelihood function of  $\eta$ .

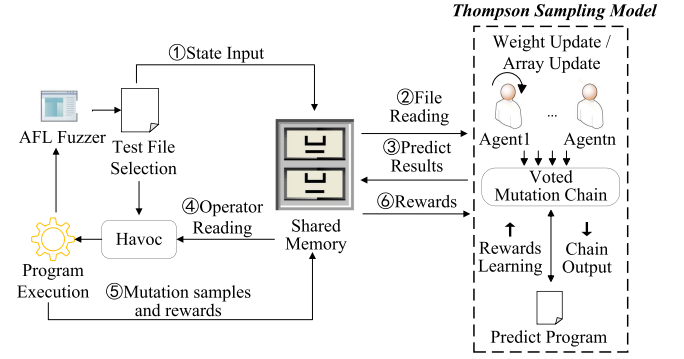


Fig. 5. Interaction process between AFL and TS.

Based on (15), the posterior probability distribution of the mutation operator can be calculated as follows:

$$P(\eta_k|D_t) \propto P(\eta_k) \times P(D_t|\eta_k) = \eta^{\alpha_k-1+S[k]}(1 - \eta)^{\beta_k-1+F[k]} \quad (16)$$

where  $D_t$  and  $\eta$  are the mutation strategy-reward set in the first  $t$  rounds of mutation and the reward probability parameter for each mutation strategy, respectively.

Based on (16), the posterior distribution of the AFL mutation strategy selection can be calculated as follows:

$$\eta_k|D_t \sim \text{beta}(\alpha_k + S[k], \beta_k + F[k]) \quad (17)$$

where beta represents the beta distribution,  $k$  is the  $k$ th mutation strategy and  $t$  is the  $t$ th iteration round.

According to (13)–(17), the parameters  $\alpha$  and  $\beta$  can be updated as follows:

$$(\alpha_{k+1}, \beta_{k+1}) = (\alpha_k, \beta_k) + (r_t, 1 - r_t) \quad (18)$$

where  $t$  is the  $t$ th action and  $k$  is the mutation round.  $r$  is the reward with the value of 0 or 1, indicating an ineffective or effective mutated sample. The interaction between the AFL and the TS model is illustrated in Fig. 5.

By utilizing a multiagent decision-making method, an update function is designed to enhance the robustness of the TS algorithm. By introducing multiagents, the TS model learns the rewards from the mutation process, and the final output of the improved TS algorithm is the decision made by the following agents' voting:

$$a_c = \arg \max_a \sum_{i=1}^T \omega_i \times \delta_i(a) \quad (19)$$

where  $a_c$  is the action selected, and  $i$  is the  $i$ th agent.  $\omega$  is the weight of the  $i$ th agent, and  $T$  is the number of the agents. The optimization process of the improved TS algorithm for AFL is shown in Algorithm 3.

## V. EVALUATION

To verify the effectiveness of DPRFUZZ, this section evaluates the performance of DQN, TRPO, and improved TS on optimizing AFL. Seven sets are set up of comparative experiments,

TABLE I  
EXPERIMENTAL ENVIRONMENT

| Experimental Environment | Configuration Information              |
|--------------------------|----------------------------------------|
| Operating System         | Ubuntu 20.04.6 LTS                     |
| Memory                   | 100G                                   |
| CPU                      | AMD Ryzen 5 4600H with Radeon Graphics |
| AFL                      | 2.52b                                  |
| Compile Environment      | GCC-9.3.0                              |
| binutils                 | 2.27                                   |
| tcpdump                  | 4.6.2                                  |
| tiff                     | 4.2.0                                  |
| zlib                     | 1.2.11                                 |
| libxml2                  | 2.9.12                                 |

including original AFL (AFL-original), AFL++, AFL-Smart, AFL optimized based on DQN (AFL-DQN), AFL optimized based on TRPO (AFL-TRPO), AFL based on joint optimization of DQN and improved TS algorithm (AFL-DT), and AFL based on joint optimization of TRPO and improved TS algorithm (AFL-TT). This section will compare different metrics of AFL and our proposed DPRFUZZ tested on different programs within 24 h.

### A. Experimental Setup

Experiments are conducted in the compile environment and the test programs' versions are listed in Table I. Three programs, *readelf*, *nm*, and *objdump*, from the Binutils toolset, are selected as experimental data. To verify the mutation operator and mutation strategy chain prediction accuracy of the proposed method, the experiments are also conducted on the other five commonly used programs, including *tcpdump*, *jpegtran*, *tiffsplit*, *minigzip*, and *xmllint*.

In the experiments, the loss function value curves are compared across different models to ascertain the optimal parameters during the experimental process. The figure of the model's loss function value is shown in Fig. 6.

Based on the loss function values, the model is deployed in accordance with the source code of AFL and the parameters are fine-tuned. In the experiments, since the action input of the RL model can only be numerical, numbers are assigned to the mutation strategy and mutation position and they are concatenated to obtain  $a = (o, p) = o \times 16^n + p$ . Here,  $n = 4$  is the offset and  $p$  takes values ranging from 0 to 4095. Since parameters are passed between the model and AFL through hexadecimal strings, the offset  $n$  is set to combine the strategy and position for AFL to process the mutation operators, such that the first half of the hexadecimal string represents the mutation strategy, and the second half represents the mutation position.

The experiment procedure is as follows. First, a number of programs under test are collected and compiled using *afl-gcc*. Subsequently, the 'make' command is employed for building the programs. AFL uses specific commands to conduct fuzzing on these programs under test. Finally, the results are collected and visualized. After compiling and instrumenting the test programs' source code, fuzzing is executed and four comprehensive metrics are defined to evaluate the effectiveness of the proposed method.

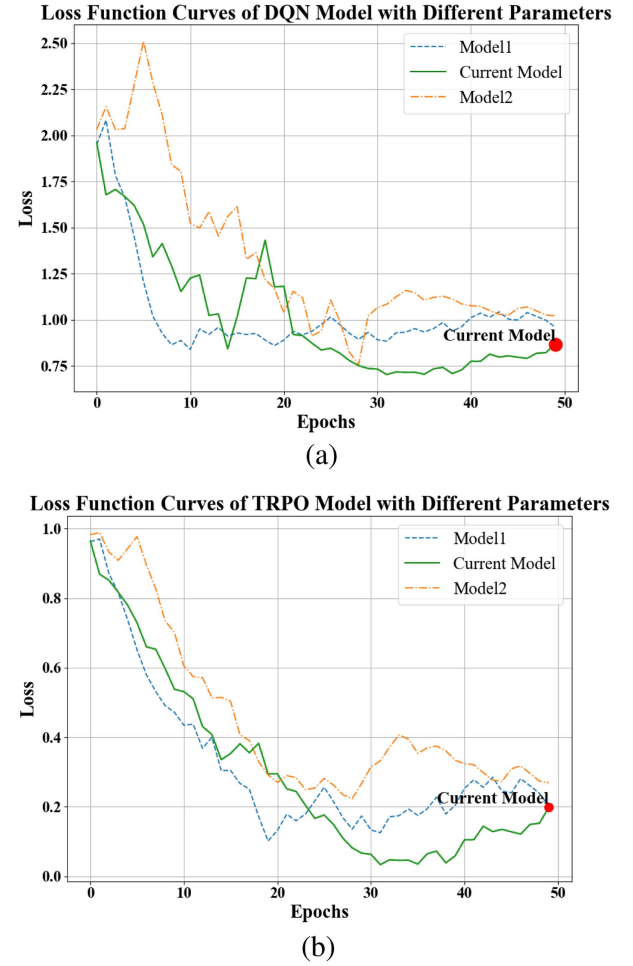


Fig. 6. Loss function value. (a) DQN loss function value curve. (b) TRPO loss function value curve.

- 1) *Effective Proportion of Mutation Strategy and Position Pairs (EPP)*: EPP is the proportion of effective mutated samples among all generated mutated samples. The precision is  $10^{-6}$ . Higher EPP indicates the predicted mutation positions and mutation strategies with higher accuracy, leading to better fuzzing results.
- 2) *Edge Path Sample Numbers (EPN)*: EPN represents the number of mutated samples that increase the coverage of edge paths and indicates the AFL's ability to trigger new paths. It can reflect the level of coverage achieved. A higher EPN indicates a higher coverage during the mutation process, enhancing the likelihood of detecting potential vulnerabilities.
- 3) *Input Gain (IG)*: IG represents the total number of valid mutated samples generated during the mutation process [23]. A larger IG indicates the generation of more effective mutated samples, implying that more execution paths of the tested program are triggered, and thus the fuzzing analysis of the tested program is more thorough.
- 4) *Crash Numbers (CN)*: CN is the number of crashes which is caused by generated mutated samples. By analyzing the mutated samples that trigger crashes, potential risks can

TABLE II

EPP OF DIFFERENT MODELS. BOLD VALUES INDICATE THE MAXIMUM EPP VALUES AMONG DIFFERENT MODELS UNDER A GIVEN DATASET WITHIN A SPECIFIC MUTATION STAGE.

| Method/Operation       | bitflip 1/1 | bitflip 2/1 | bitflip 4/1 | bitflip 8/8 | bitflip 16/8 | bitflip 32/8 | arith 8/8    | arith 16/8  | arith 32/8  | interest 8/8 | interest 16/8 | interest 32/8 | Havoc       |
|------------------------|-------------|-------------|-------------|-------------|--------------|--------------|--------------|-------------|-------------|--------------|---------------|---------------|-------------|
| <i>READELF dataset</i> |             |             |             |             |              |              |              |             |             |              |               |               |             |
| AFL                    | 0.10        | 0.35        | 0.15        | 0.05        | 0.31         | 0.09         | 0.19         | 0.00        | 0.01        | 0.11         | 0.02          | 0.03          | 0.22        |
| AFL++                  | 0.45        | 0.54        | 0.23        | 0.09        | <b>0.42</b>  | 0.09         | 1.03         | 0.01        | 0.01        | 0.22         | 0.17          | 0.10          | 0.34        |
| AFL-DQN                | 0.79        | 0.38        | 0.20        | 0.08        | 0.31         | 0.06         | 2.00         | 0.06        | 0.06        | 0.06         | 0.27          | 0.68          | 0.12        |
| AFL-TRPO               | 1.56        | 0.43        | 0.28        | 0.09        | 0.35         | 0.08         | 2.54         | 0.09        | 0.08        | 0.07         | 0.30          | 0.89          | 0.13        |
| AFL-DT                 | 1.93        | <b>0.89</b> | 0.32        | 0.26        | 0.11         | 0.09         | 4.55         | 0.51        | <b>0.91</b> | 0.23         | 0.75          | 1.76          | <b>0.36</b> |
| AFL-TT                 | <b>2.00</b> | 0.87        | <b>0.35</b> | <b>0.34</b> | 0.34         | <b>0.23</b>  | <b>7.75</b>  | <b>0.57</b> | 0.13        | <b>0.36</b>  | <b>0.93</b>   | <b>1.77</b>   | 0.20        |
| <i>OBJDUMP dataset</i> |             |             |             |             |              |              |              |             |             |              |               |               |             |
| AFL                    | 3.99        | 1.10        | 0.51        | 0.97        | 1.43         | 1.57         | 0.34         | 0.05        | 0.02        | 0.83         | 0.25          | 0.12          | 2.41        |
| AFL++                  | 4.51        | 1.44        | 0.89        | 1.47        | 1.67         | 1.70         | 0.44         | 0.45        | 0.12        | 1.01         | 0.54          | 0.17          | 2.56        |
| AFL-Smart              | 5.03        | 1.78        | 1.04        | 1.75        | 2.03         | 1.84         | 0.79         | 0.66        | 0.20        | 1.00         | 0.49          | 0.26          | 2.83        |
| AFL-DQN                | <b>6.71</b> | 2.37        | 1.03        | 3.11        | 1.61         | 1.84         | 22.32        | 5.14        | 1.44        | 2.20         | 4.19          | <b>6.91</b>   | 2.27        |
| AFL-TRPO               | 6.18        | 2.22        | <b>1.37</b> | 1.75        | 2.98         | 2.85         | 22.14        | 4.23        | <b>1.52</b> | 2.44         | 4.96          | 6.71          | 2.34        |
| AFL-DT                 | 6.00        | 1.76        | 1.01        | 1.80        | <b>5.23</b>  | 2.28         | 23.31        | <b>5.22</b> | 1.29        | 2.89         | 4.78          | 6.37          | <b>2.99</b> |
| AFL-TT                 | 6.40        | <b>2.53</b> | 1.36        | <b>2.00</b> | 3.05         | <b>3.00</b>  | <b>22.34</b> | 3.57        | 1.46        | <b>3.02</b>  | <b>5.35</b>   | 6.49          | 2.59        |
| <i>NM dataset</i>      |             |             |             |             |              |              |              |             |             |              |               |               |             |
| AFL                    | 1.55        | 0.35        | 0.08        | 0.31        | <b>1.30</b>  | 1.01         | 0.19         | 0.01        | 0.00        | 0.26         | 0.01          | 0.06          | 0.16        |
| AFL++                  | 1.79        | 0.34        | 0.14        | 0.56        | 1.01         | <b>1.05</b>  | 0.24         | 0.09        | 0.00        | 0.24         | 0.08          | 0.11          | 0.32        |
| AFL-Smart              | 2.03        | 0.57        | 0.30        | <b>0.88</b> | 1.23         | 0.99         | 0.36         | 0.14        | 0.05        | 0.32         | 0.09          | 0.44          | 0.43        |
| AFL-DQN                | 1.27        | 0.37        | 0.13        | 0.31        | 0.05         | 0.07         | 2.21         | 0.10        | 0.02        | 0.21         | 0.22          | 0.22          | 0.21        |
| AFL-TRPO               | 1.43        | 0.61        | 0.19        | 0.42        | 0.40         | 0.16         | 2.78         | 0.25        | 0.27        | 0.16         | 0.67          | 1.37          | 0.25        |
| AFL-DT                 | 2.13        | 0.88        | <b>0.36</b> | 0.23        | 0.08         | 0.21         | 4.23         | 0.10        | 0.13        | 0.08         | 0.57          | 0.69          | 0.32        |
| AFL-TT                 | <b>3.34</b> | <b>0.89</b> | 0.31        | 0.87        | 0.77         | 0.82         | <b>8.35</b>  | <b>1.09</b> | <b>0.37</b> | <b>0.45</b>  | <b>0.88</b>   | <b>1.82</b>   | <b>0.49</b> |
| <i>tcpdump dataset</i> |             |             |             |             |              |              |              |             |             |              |               |               |             |
| AFL                    | 4.86        | 1.09        | 0.53        | 0.43        | 0.45         | 0.08         | 0.47         | 0.04        | 0.01        | 1.08         | 0.32          | 1.93          | 0.89        |
| AFL++                  | 4.91        | 1.83        | 0.73        | 0.85        | 0.77         | 0.13         | 0.56         | 0.10        | 0.01        | 1.45         | 0.64          | 1.94          | 1.01        |
| AFL-Smart              | 6.04        | 2.34        | 0.64        | 0.94        | 0.60         | 0.16         | 1.03         | 0.22        | 0.03        | 1.07         | 0.66          | 2.00          | 1.47        |
| AFL-DQN                | 6.87        | 2.99        | 2.24        | 1.45        | 1.55         | 1.64         | 38.37        | 5.82        | 1.58        | 7.02         | 17.08         | 13.05         | 1.21        |
| AFL-TRPO               | 6.68        | 3.17        | <b>2.73</b> | 1.56        | <b>1.64</b>  | 1.84         | 38.67        | <b>6.20</b> | 1.44        | 6.64         | 17.16         | 13.66         | 1.22        |
| AFL-DT                 | 6.86        | <b>3.71</b> | 2.72        | 1.59        | 1.48         | 1.83         | 44.81        | 5.64        | <b>1.83</b> | <b>7.82</b>  | <b>17.48</b>  | 15.62         | 1.37        |
| AFL-TT                 | <b>7.09</b> | 3.43        | 2.46        | <b>1.60</b> | 1.41         | <b>2.16</b>  | <b>47.06</b> | 5.86        | 1.58        | 6.47         | 13.58         | <b>20.46</b>  | <b>1.58</b> |

be uncovered in the program and assist in vulnerability remediation.

TABLE III  
CRASH NUMBERS

| objdump   |    | tcpdump   |     |
|-----------|----|-----------|-----|
| AFL       | 6  | AFL       | 73  |
| AFL++     | 7  | AFL++     | 144 |
| AFL-Smart | 9  | AFL-Smart | 172 |
| AFL-DQN   | 7  | AFL-DQN   | 132 |
| AFL-TRPO  | 8  | AFL-TRPO  | 205 |
| AFL-DT    | 9  | AFL-DT    | 134 |
| AFL-TT    | 11 | AFL-TT    | 312 |

### B. Accuracy of Valid Mutation Operator Prediction

As shown in Table II, DPRFUZZ achieves higher EPP compared to AFL on eight datasets. For *nm*, AFL-TT achieves the highest EPP for 11 mutation operations compared to other models, while AFL only reaches the optimal EPP for 2 mutation operations. On average, AFL-TT improves by 1.17 compared to AFL for 13 operations. Furthermore, AFL-TT achieves the highest EPP of 8.35 for the arith 8/8 mutation operation, representing an increase of 8.16 compared to AFL-original. This demonstrates that DPRFUZZ can accurately predict effective mutation operators, leading to triggering new paths and causing crashes. It is also observed that AFL-TT and AFL-DT exhibit superior performance and gain the highest EPP 47.06 and 44.81 for *tcpdump*, this is attributed to the joint optimization using the improved TS algorithm, which enhances AFL's global search capability and provides a high-quality experience repository for training DQN and TRPO.

### C. Performance in Branch Discovery

As can be seen from Fig. 7, in the initial stages of the experiment, the EPNs of the proposed models for *readelf* and *objdump* are slightly lower than those of the AFL-original. This is owing to the RL models are not converged and the prediction accuracy is relatively low. After a period of learning, the prediction accuracy increases, resulting in DPRFUZZ achieving overall higher EPNs compared to the AFL-original. For the 24-hour *tcpdump* testing, AFL-TT results in the maximum EPN value of 3326, with an increase of 24.2% compared to the AFL-original. Furthermore, all of our proposed models demonstrate superior performance, and the most notable improvement is observed in *tiffsplit* with an

EPN increase of 87.2% compared to AFL. This implies that the RL models can effectively improve the edge coverage of AFL by predicting high path triggering rate mutation operators.

### D. Performance of Valid Input Gain

As shown in Fig. 7, for the *tcpdump* testing, AFL-TT discovers the highest number of valid mutated samples, with an IG value of 8537, representing an increase of 3421 compared to AFL-original. Moreover, the IG values are significantly improved across different test programs in our optimization models, with the most notable improvement observed in *tcpdump* which increases by 76.7%. It can be concluded that DPRFUZZ can enhance the depth of vulnerability mining and achieve more comprehensive fuzzing ability.

### E. Crashes

As can be concluded from Table III, the proposed DPRFUZZ generates more mutated samples that can trigger potential program crashes. Notably, for *tcpdump*, the AFL-TT model

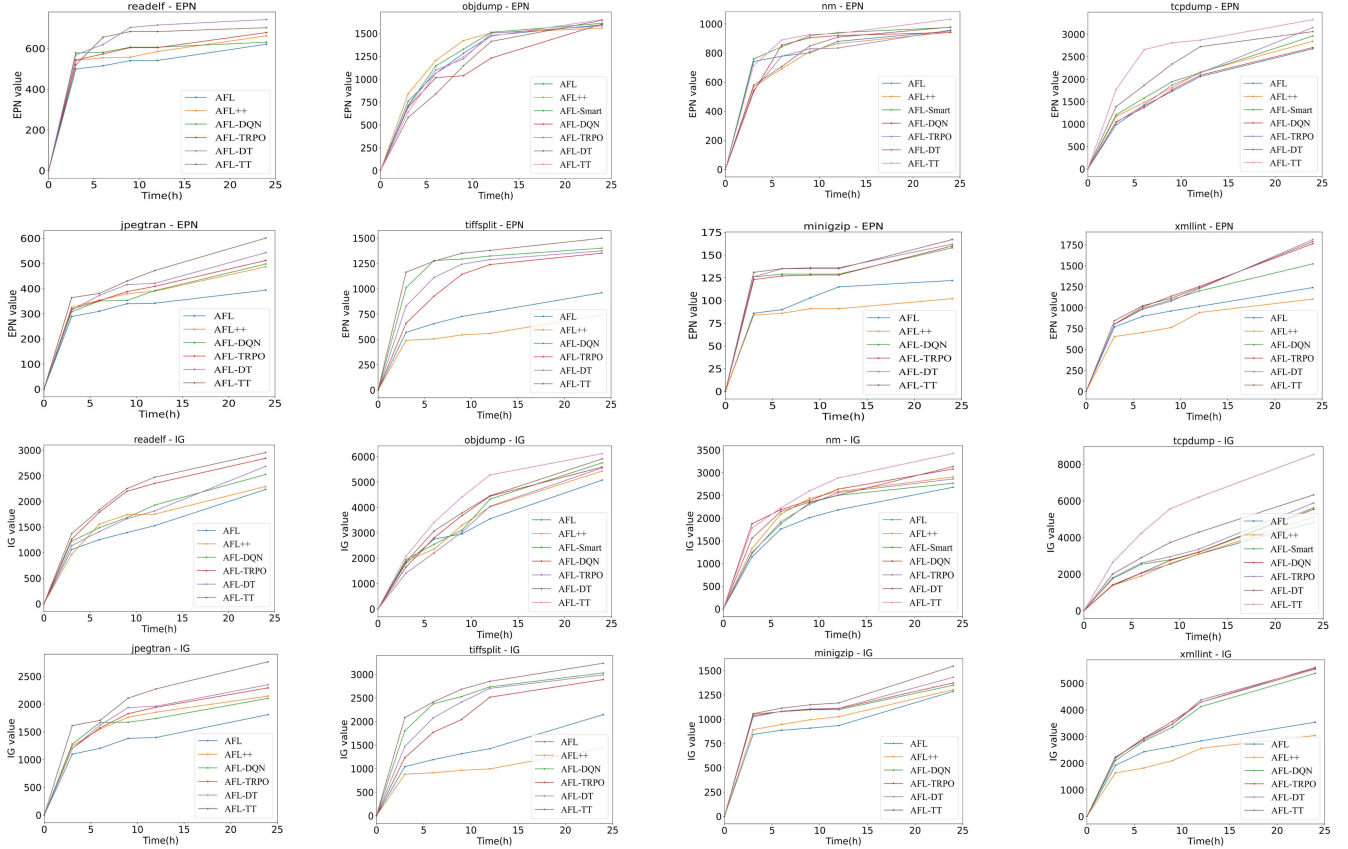


Fig. 7. Experimental result.

achieves the most prominent improvement, with an increase of 239 crashes. The efficacy of using TRPO to uncover crashes is superior to that of using DQN, with an average increase of 94.1% in the number of crashes. By analyzing the crashes DPRFUZZ gains, deeper vulnerabilities can be uncovered in the program.

## F. Discussion

From Fig. 7, it can be seen that DPRFUZZ conspicuously outperforms the original AFL, and the performance of the four models can be ranked as  $\text{AFL-TT} > \text{AFL-DT} > \text{AFL-TRPO} > \text{AFL-DQN}$ . It is observed that AFL employing a joint optimization model outperforms AFL utilizing only a single model optimization. This is attributed to the utilization of the TS model, which predicts effective mutation strategy chains during the Havoc mutation phase, generating more mutated samples that can trigger new paths. In addition, AFL-TRPO consistently outperforms AFL-DQN. This is because the TRPO utilizes a trust region constraint method to limit policy changes, which facilitates the generation of additional effective mutated samples. Also, TRPO improves the utilization efficiency of the experience repository and constraining policy changes, making it more likely for DPRFUZZ to generate mutated samples that trigger crashes.

Although integrating the RL models into AFL offers significant potential for improving fuzzing performance, it also

introduces substantial challenges. The training time required for the RL models and the associated computational overhead can be prohibitive under resource-constrained environments. Therefore, the computational overhead experiments are designed and the results are compared with AFL, ensuring a comprehensive evaluation of the proposed approach. The computational overhead of the models can be seen in Fig. 8. It is observed that the additional time consumed for a single mutation round is consistently within 1.5 s, which indicates a relatively low time cost for learning and prediction processes in the models. Furthermore, when comparing the efficiency of predicting effective mutation operators, it is evident that over a 24-hour period, AFL-TT outperforms AFL-original by approximately 3 effective mutation operators per minute on average. This suggests that under the given vulnerability exploitation scenario, the computational cost of the model is acceptable.

Despite the advancements achieved by DPRFUZZ, it is important to acknowledge certain limitations that can impact its performance in specific scenarios. One significant challenge is encountered when testing complex programs, where the model occasionally falls into local optima. The main reason is that the RL models may overly focus on exploiting known mutation strategies, while neglecting to explore better mutation strategies. A feasible solution is to introduce the enhanced exploration techniques, which encourage the RL models to explore less visited states and actions, increasing the chances of discovering

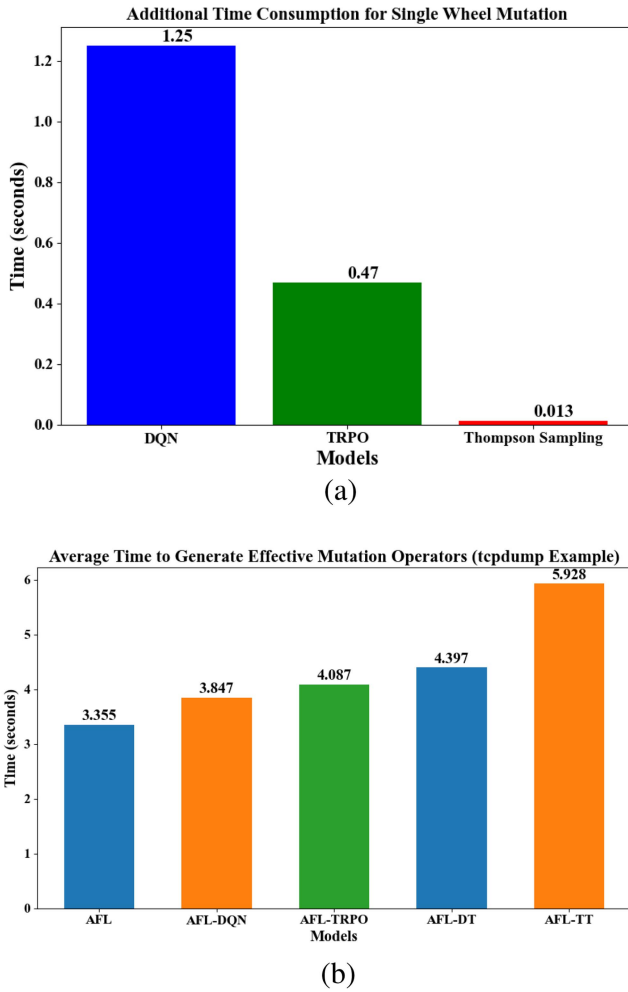


Fig. 8. Computational overhead experiment outcomes. (a) Additional time consumption for single mutation round. (b) Average time to generate effective mutation operators.

global optima [40]. Another challenge is that the model may struggle to be effectively trained in sparse data environments, leading to suboptimal prediction outcomes. Sparse data results in insufficient information for the RL models to accurately predict different action values, and hinders the optimization process. A feasible solution is to integrate transfer learning techniques, which improve the generalization of the model by leveraging knowledge gained from related domains [41].

### G. Vulnerabilities Mining

The practical application effect of DPRFUZZ is validated in the vulnerability mining realm. It is found that DPRFUZZ is capable of identifying all the vulnerabilities that AFL could detect, which confirms the robustness of our optimization. The comparison of vulnerability mining ability between AFL and DPRFUZZ can be seen from the Table IV.

Moreover, DPRFUZZ discovers five zero-day vulnerabilities that remain undetected previously. The new memory leak vulnerability in *apng2gif* can lead to system crashes and performance degradation in applications that frequently convert

TABLE IV  
VULNERABILITIES MINING

| Program | Identifier       | Type               | AFL   |         | DPRFUZZ |         |
|---------|------------------|--------------------|-------|---------|---------|---------|
|         |                  |                    | Found | Time(h) | Found   | Time(h) |
| readelf | CVE-2019-14444   | overflow           | ✓     | 9       | ✓       | 7       |
|         | CVE-2019-8906    | access violation   | ✓     | 12      | ✓       | 12      |
|         | CVE-2019-8904    | buffer reread      | ✗     | ✗       | ✓       | 10      |
|         | CVE-2017-6965    | overflow           | ✓     | 4       | ✓       | 2       |
| tcpdump | CVE-2019-15167   | memory leak        | ✗     | ✗       | ✓       | 7       |
|         | CVE-2018-16229   | access violation   | ✓     | 7       | ✓       | 8       |
|         | CVE-2018-14879   | access violation   | ✓     | 9       | ✓       | 6       |
| objdump | CVE-2023-22606   | heap overflow      | ✗     | ✗       | ✓       | 11      |
|         | CVE-2018-20671   | overflow           | ✓     | 7       | ✓       | 7       |
|         | CVE-2018-1000876 | overflow           | ✓     | 9       | ✓       | 8       |
|         | CVE-2018-12699   | overflow           | ✓     | 6       | ✓       | 3       |
|         | CVE-2017-7224    | out-of-bound write | ✗     | ✗       | ✓       | 7       |
| xmllint | AVD-2021-3516    | access violation   | ✓     | 12      | ✓       | 10      |

APNG images to GIF format. Also, these new allocator out of memory vulnerabilities in *qpdf* can cause application crashes and data corruption when processing encrypted, multipage, or rotated PDF files, respectively. The discovery of these zero-day vulnerabilities not only underscores the superiority of DPRFUZZ in terms of vulnerability mining, but also demonstrates the potential of our technique to contribute to software security. Readers with an interest in these zero-day vulnerabilities can refer online.<sup>1</sup>

## VI. CONCLUSION

In this article, we study the negative impact of randomness and traversal in coverage-guided fuzzers. A fuzzing optimization solution DPRFUZZ is proposed, which predicts more effective mutation operators and mutation strategy chains, improving the efficiency and effectiveness of the state-of-art fuzzer AFL. We employ the RL models to drive AFL directly toward nonexplored paths. Experiments show that our proposed DPRFUZZ is effective and efficient for path discovery and crash finding. It is also discovered that the joint optimization of DQN/TRPO and TS algorithm gains a better performance compared to the single ones. Moreover, DPRFUZZ also shows better crash mining ability with 312 crashes, and the maximum IG value improvement in 24 h reaches 2026. DPRFUZZ also shows good effect in industry area, and it successfully discovers five zero-day vulnerabilities in actual applied software. For readers interested in the details of DPRFUZZ, the corresponding source code is available online.<sup>2</sup> In the future, we plan to introduce the enhanced exploration techniques and the transfer learning techniques to better adapt to the challenges.

## REFERENCES

- [1] S. Gan et al., "Path sensitive fuzzing for native applications," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1544–1561, May/Jun. 2022.
- [2] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, pp. 1–36, Sep. 2022.
- [3] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.

<sup>1</sup>[Online]. Available: <https://github.com/p3n20wn/fuzz>

<sup>2</sup>[Online]. Available: <https://github.com/Lastfarewell12/RLFuzz>

- [4] Y. Li, J. Wu, G. Yu, and D. Zhao, "Improved group sparse modal decomposition methods with applications to fault diagnosis of rotating machinery," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8102–8114, Jun. 2024.
- [5] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, and Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14595–14620, Aug. 2023.
- [6] C. Zhou, M. Wang, J. Liang, Z. Liu, and Y. Jiang, "Zeror: Speed up fuzzing with coverage-sensitive tracing and scheduling," in *Proc. 2020 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 858–870.
- [7] V. J. Manès et al., "The art, science, and engineering of fuzzing: A survey," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2312–2331, Nov. 2021.
- [8] B. Wang, R. Wang, and H. Song, "Toward the trustworthiness of industrial robotics using differential fuzz testing," *IEEE Trans. Ind. Inform.*, vol. 19, no. 3, pp. 2782–2791, Mar. 2023.
- [9] A. Fioraldi, A. Mantovani, D. Maier, and D. Balzarotti, "Dissecting american fuzzy lop: A fuzzbench evaluation," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, pp. 1–26, Mar. 2023.
- [10] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based grey-box fuzzing as Markov chain," *IEEE Trans. Softw. Eng.*, vol. 45, no. 5, pp. 489–506, May 2019.
- [11] S. Gan et al., "CollAFL: Path sensitive fuzzing," in *Proc. 2018 IEEE Symp. Secur. Privacy*, 2018, pp. 679–696.
- [12] Y. Song, J. Lu, H. Lu, and G. Zhang, "Fuzzy clustering-based adaptive regression for drifting data streams," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 3, pp. 544–557, Mar. 2020.
- [13] Y. Li et al., "G-Fuzz: A directed fuzzing framework for gVisor," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 168–185, Jan./Feb. 2024.
- [14] J. Li, B. Zhao, and C. Zhang, "Fuzzing: A survey," *Cybersecurity*, vol. 1, no. 1, 2018, Art. no. 6.
- [15] S. Li et al., "Deep learning for coverage-guided fuzzing: How far are we?," *IEEE Trans. Dependable Secure Comput.*, early access, Sep. 9, 2022, doi: 10.1109/TDSC.2022.3200525.
- [16] G. D. Ath, R. M. Everson, A. A. M. Rahat, and J. E. Fieldsend, "Greed is good: Exploration and exploitation trade-offs in Bayesian optimisation," *ACM Trans. Evol. Learn. Optim.*, vol. 1, pp. 1–22, Apr. 2021.
- [17] D. Wijayasekara, O. Linda, M. Manic, and C. Rieger, "Mining building energy management system data using fuzzy anomaly detection and linguistic descriptions," *IEEE Trans. Ind. Inform.*, vol. 10, no. 3, pp. 1829–1840, Aug. 2014.
- [18] B. Yang, S. Hong, H. Wang, and D. Zhao, "Complex dynamic slow independent symbol aggregation approximation for multimodal temporal fuzzy cognitive map," *IEEE Trans. Ind. Inform.*, vol. 20, no. 11, pp. 12882–12890, Nov. 2024.
- [19] Y. Xue et al., "xFuzz: Machine learning guided cross-contract fuzzing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 515–529, Mar./Apr. 2024.
- [20] P. Zhang, B. Ren, H. Dong, and Q. Dai, "CAGFuzz: Coverage-guided adversarial generative fuzzing testing for image-based deep learning systems," *IEEE Trans. Softw. Eng.*, vol. 48, no. 11, pp. 4630–4646, Nov. 2022.
- [21] H. Dai, C.-A. Sun, H. Liu, and X. Zhang, "DFuzzer: Diversity-driven seed queue construction of fuzzing for deep learning models," *IEEE Trans. Rel.*, vol. 73, no. 2, pp. 1075–1089, Jun. 2024.
- [22] X. Zhang et al., "Duo: Differential fuzzing for deep learning operators," *IEEE Trans. Rel.*, vol. 70, no. 4, pp. 1671–1685, Dec. 2021.
- [23] L. Yang et al., "Seq2Seq-AFL: Fuzzing via sequence-to-sequence model," *Int. J. Mach. Learn. Cybern.*, vol. 15, pp. 4403–4421, Oct. 2024.
- [24] Y. Yan, G. Li, Y. Chen, and J. Fan, "The efficacy of pessimism in asynchronous Q-learning," *IEEE Trans. Inf. Theory*, vol. 69, no. 11, pp. 7185–7219, Nov. 2023.
- [25] H. Liang, X. Yu, X. Cheng, J. Liu, and J. Li, "Multiple targets directed greybox fuzzing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 325–339, Jan./Feb. 2024.
- [26] P. Li, W. Meng, and C. Zhang, "SDFuzz: Target states driven directed fuzzing," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USA: USENIX Assoc., Aug. 2024, pp. 2441–2457.
- [27] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13344–13362, Nov. 2023.
- [28] Z. Li and C. Guo, "Multi-agent deep reinforcement learning based spectrum allocation for D2D underlay communications," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1828–1840, Feb. 2020.
- [29] M. Wei, Y. Huang, J. Yang, J. Wang, and S. Wang, "CoCoFuzzing: Testing neural code models with coverage-guided fuzzing," *IEEE Trans. Rel.*, vol. 72, no. 3, pp. 1276–1289, Sep. 2023.
- [30] Z. Gao et al., "FA-Fuzz: A novel scheduling scheme using firefly algorithm for mutation-based fuzzing," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 1–15, Jan. 2024.
- [31] X. Zhou et al., "UltraFuzz: Towards resource-saving in distributed fuzzing," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2394–2412, Apr. 2023.
- [32] W. Li, S. Dou, Y. Wu, C. Li, and Y. Liu, "COCL: An intelligent framework for enhancing deep learning-based vulnerability detection," *IEEE Trans. Ind. Inform.*, vol. 20, no. 3, pp. 4953–4961, Mar. 2024.
- [33] D.-L. Lin, Y. Zhang, H. Ren, B. Khailany, S.-H. Wang, and T.-W. Huang, "GenFuzz: GPU-accelerated hardware fuzzing using genetic algorithm with multiple inputs," in *Proc. 2023 60th ACM/IEEE Des. Automat. Conf.*, 2023, pp. 1–6.
- [34] X. Zhu et al., "CSI-Fuzz: Full-speed edge tracing using coverage sensitive instrumentation," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 912–923, Mar./Apr. 2022.
- [35] B. Hazarika and K. Singh, "AFL-DMAAC: Integrated resource management and cooperative caching for URLLC-IoV networks," *IEEE Trans. Intell. Veh.*, vol. 9, no. 6, pp. 5101–5117, Jun. 2024.
- [36] S. Zhu et al., "Better pay attention whilst fuzzing," *IEEE Trans. Softw. Eng.*, vol. 50, no. 2, pp. 190–208, Feb. 2024.
- [37] J. Eom, S. Jeong, and T. Kwon, "Fuzzing Javascript interpreters with coverage-guided reinforcement learning for LLM-based mutation," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, New York, NY, USA: Assoc. Comput. Machinery, 2024, pp. 1656–1668.
- [38] M. Tappler, A. Pferscher, B. K. Aichernig, and B. Könighofer, "Learning and repair of deep reinforcement learning policies from fuzz-testing data," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, New York, NY, USA: Assoc. Comput. Machinery, 2024, pp. 38–50.
- [39] C. Im, S. Ahn, and D. Yoon, "Modulation classification based on Kullback-Leibler divergence," in *Proc. 2020 IEEE 15th Int. Conf. Adv. Trends Radioelectronics, Telecommun. Comput. Eng.*, 2020, pp. 373–376.
- [40] C. Zhou et al., "Towards better semantics exploration for browser fuzzing," *Proc. ACM Program. Languages*, vol. 7, pp. 604–631, Oct. 2023.
- [41] L. Xi et al., "Enhancing coverage in stateful protocol fuzzing via value-based selection," in *Proc. 2024 IEEE Symp. Comput. Commun.*, 2024, pp. 1–6.



**Liqun Yang** received the Ph.D. degree in information security from the School of Computer Science and Engineering, Beihang University, in China.

Since 2021, he has been with Beihang University, where he is currently an Associate Professor with the School of Cyber Science and Technology. His research interests include cyber security, IoT security, ICS security, protocol fuzzing, and applied privacy.



**Ruihao Li** is currently working toward the Master's degree in network and information security with the Beijing University of Posts and Telecommunications, Beijing, China.

His research interests include fuzzing, traffic fingerprinting for security, and firmware vulnerability analysis for bolstering the security of network and embedded systems.



**Chaoren Wei** is currently working toward the Master's degree in electronic information with Beihang University, Beijing China.

His research interests include the application of artificial intelligence (AI) in fuzz testing, aiming to enhance the intelligence level of software security testing.



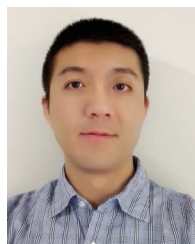
**Jian Yang** received his B.S. degree and Ph.D. in computer science from Beihang University (Beijing, China).

He is an Associate Professor (Tenure-Track) with Beihang University, Beijing, China. Previously, he was a Researcher in Alibaba Group, Qwen team, where his primary research areas include multilingual natural language understanding and generation, pretraining and fine-tuning for large language model, software code understanding and generation, and software fuzzing. During his tenure with Alibaba (as a Member of the Alibaba Star Program), he was a core contributor to Qwen and QwenCoder, responsible for the code capabilities of the Qianwen model and the code-specific models CodeQwen1.5/Qwen2.5-Coder. He actively participated in open-source initiatives with the community and universities. He published multiple papers in top-tier conferences, such as ICLR, NeurIPS, ICML, ACL, EMNLP, NAACL, COLING, WWW, AAAI, IJCAI, and WMT with over 6500+ citations on Google Scholar. During his Ph.D., he was deeply involved in academic-industrial collaboration, participating in a long-term joint Ph.D. program between Beihang University and Microsoft Research Asia.

Dr. Yang was the recipient of first place in three tracks of the multilingual translation competition at WMT 2021 (World Machine Translation, involving tens of thousands of translation directions). He served as an Area Chair for conferences, such as ACL, EMNLP, NAACL, and NeurIPS.



**Yuze Yang** received the M.S. degree in computer technology from the School of Cyber Engineering, Xidian University, China, in 2022. He is currently pursuing a Ph.D. degree in Cyberspace Security at the School of Cyber Science and Technology, Beihang University, China. His research interests include cryptocurrency, homomorphic encryption, searchable encryption, and applied cryptography.



**Liang Sun** received Ph.D. degree in electrical engineering from the Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2010.

Since 2016, he is an Associate Professor with the Beihang University, Beijing, China. From 2010 to 2013, he was a Researcher with the Alcatel-Lucent Shanghai Bell and the NEC Corporation. From September 2014 to December 2015, he was a Postdoctoral Research Fellow with the Department of Electrical and Computer Engineering, the University of British Columbia, Vancouver, BC, Canada. He also held Visiting positions with the Waseda University, Japan, and the South University of Science and Technology of China, Shenzhen, China. His research interests include information theory and signal processing; in particular design and analysis of wireless communication systems, MIMO, physical layer security, ultra-dense heterogeneous networks, Internet of Things, and ultra-reliable low-latency communications.

Dr. Sun was the recipient of 2010 Young Author Best Paper Award by the IEEE Signal Processing Society.



**Dong Zhao** (Senior Member, IEEE) received the B.E. degree in automation and the Ph.D. degree in control science and engineering from the Beijing University of Chemical Technology, Beijing, China, in 2011 and 2016, respectively.

Since 2022, he has been a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. From 2017 to 2021, he was a Postdoctoral Research Fellow with the Institute for Automatic Control and Complex Systems (AKS), University of Duisburg-Essen, Duisburg, Germany, and the KIOS Research and Innovation Center of Excellence with the University of Cyprus, respectively. His research interests include fault diagnosis, fault-tolerant control, cyber-physical systems, and cyber security.



**Zhoujun Li** received the B.S. degree from Wuhan University, Wuhan, China, in 1984, and the M.S. and Ph.D. degrees from the National University of Defense Technology, Changsha, China, in 1986 and 1999, respectively, all in computer science.

He is currently a Professor with Beihang University, Beijing, China. His research interests include information security.